

BÁO CÁO NIÊN LUẬN

XÂY DỰNG WEBSITE KIỂM TRA TOÁN TRỰC TUYẾN HỖ TRỢ TỰ ĐỘNG LƯU VÀ KHÔI PHỤC BÀI LÀM

Tên sản phẩm: JinMath

Loại sản phẩm: Website kiểm tra Toán trực tuyến

Công nghệ chính: Node.js, Express.js, EJS, MySQL 8+

Báo cáo này mô tả phiên bản V1 của JinMath, được xây dựng cho mục đích học tập và niên luận. Các nội dung kỹ thuật được đối chiếu với mã nguồn, tài liệu API, sơ đồ ERD và bộ kiểm thử đi kèm dự án.

LỜI CẢM ƠN

Em xin chân thành cảm ơn quý thầy cô đã truyền đạt kiến thức chuyên môn, phương pháp nghiên cứu và những kinh nghiệm thực tế trong quá trình học tập. Những kiến thức về phân tích thiết kế hệ thống, lập trình web, cơ sở dữ liệu và kiểm thử phần mềm là nền tảng quan trọng để em thực hiện đề tài này.

Em cũng xin cảm ơn gia đình, bạn bè và những người đã hỗ trợ, động viên em trong quá trình khảo sát yêu cầu, xây dựng và hoàn thiện sản phẩm. Dù đã cố gắng, báo cáo chắc chắn vẫn còn những thiếu sót; em mong nhận được các ý kiến góp ý để tiếp tục hoàn thiện kiến thức và sản phẩm trong tương lai.

LỜI MỞ ĐẦU

Sự phát triển của hạ tầng Internet và các nền tảng số đã làm thay đổi đáng kể hoạt động dạy học, đặc biệt là kiểm tra và đánh giá trực tuyến. Với môn Toán, một hệ thống kiểm tra không chỉ cần hiển thị câu hỏi và ghi nhận điểm số, mà còn phải hỗ trợ cách biểu diễn công thức, nhiều dạng câu hỏi và một quy trình làm bài đáng tin cậy. Trong thực tế, học sinh có thể mất kết nối, tải lại trang hoặc vô tình đóng trình duyệt khi đang làm bài. Nếu đáp án chỉ tồn tại tạm thời trên giao diện, các tình huống này có thể làm mất dữ liệu và ảnh hưởng trực tiếp đến tính công bằng của bài kiểm tra.

Từ vấn đề trên, đề tài lựa chọn xây dựng **JinMath – website kiểm tra Toán trực tuyến hỗ trợ tự động lưu và khôi phục bài làm**. Trọng tâm của hệ thống là cơ chế autosave đa tầng: đáp án được lưu trên máy chủ, lưu tạm tại trình duyệt bằng `localStorage`, và được kiểm soát phiên bản bằng `answer_version` để hạn chế request cũ ghi đè lên dữ liệu mới. Bên cạnh phòng thi, hệ thống hỗ trợ quản lý lớp học, ngân hàng câu hỏi, tạo và công bố đề, chấm điểm, công bố kết quả, xử lý sự cố và thống kê cơ bản.

Mục tiêu chung của đề tài là thiết kế và triển khai một website kiểm tra Toán hoạt động được với hai nhóm người dùng là giáo viên và học sinh. Các mục tiêu cụ thể gồm: hỗ trợ bốn loại câu hỏi; tạo được quy trình giáo viên tạo đề, giao đề và chấm bài; cho phép học sinh làm bài, lưu đáp án tự động, khôi phục sau khi tải lại trang và đồng bộ lại khi có mạng; áp dụng quy tắc thời gian theo máy chủ; và kiểm thử các tình huống cạnh tranh, mất kết nối hoặc hết giờ.

Phạm vi của đề tài là phiên bản V1 dành cho mô hình một giáo viên chủ hệ thống cùng các lớp học của mình. Vai trò `GIAO_VIEN` được tạo sẵn trong dữ liệu khởi tạo, không có đăng ký giáo viên công khai; vai trò `HOC_SINH` có thể đăng ký công khai. Đề tài không triển khai nhập đề từ Word/PDF hoặc Excel, OCR, AI tạo câu hỏi hay chấm tự luận, camera giám sát, chat, ứng dụng di động, WebSocket và vai trò quản trị riêng. Chức năng quên mật khẩu qua SMTP, nếu được cấu hình, chỉ là tiện ích hỗ trợ tài khoản chứ không phải mục tiêu cốt lõi của niên luận.

Đề tài được thực hiện theo các bước: khảo sát yêu cầu; phân tích nghiệp vụ; thiết kế use case, cơ sở dữ liệu và kiến trúc; lập trình theo mô hình phân tầng; sau đó kiểm thử thủ công, kiểm thử hồi quy tự động và đánh giá kết quả. Báo cáo gồm bốn chương: Chương 1 trình bày tổng quan và công nghệ; Chương 2 phân tích, thiết kế; Chương 3 mô tả quá trình xây dựng; Chương 4 trình bày kiểm thử và đánh giá. Phần cuối là kết luận, hướng phát triển, tài liệu tham khảo và phụ lục.

CHƯƠNG 1 — TỔNG QUAN

1.1. Giới thiệu đề tài

1.1.1. Bối cảnh kiểm tra trực tuyến trong giáo dục

Kiểm tra trực tuyến tạo điều kiện tổ chức đánh giá linh hoạt về không gian, giảm thao tác tổng hợp điểm và giúp giáo viên theo dõi tiến độ làm bài. Đối với môn Toán, môi trường số còn có lợi thế trong việc hiển thị ký hiệu, phân số, căn thức, chỉ số và các biểu thức dài một cách nhất quán. Tuy nhiên, số hóa quy trình kiểm tra không đơn thuần là thay giấy bằng màn hình. Hệ thống cần xác

định rõ thời hạn, quyền truy cập, trạng thái bài làm, cơ chế chấm và cách bảo vệ dữ liệu của người học.

1.1.2. Vấn đề mất dữ liệu khi thi trực tuyến

Mạng không ổn định, thao tác refresh, lỗi trình duyệt và đóng nhầm tab là những sự cố phổ biến. Một request lưu đáp án có thể đến muộn hơn request mới hơn; nếu máy chủ không phân biệt phiên bản, lựa chọn cũ có thể ghi đè lựa chọn mới. Ngoài ra, đồng hồ dựa hoàn toàn vào thời gian máy khách dễ bị sai lệch. Các vấn đề này làm giảm sự tin cậy của bài thi, dù giao diện có đầy đủ chức năng cơ bản.

1.1.3. Nhu cầu autosave và khôi phục

JinMath giải quyết vấn đề bằng ba lớp bảo vệ: dữ liệu chính trên MySQL, vùng tạm localStorage trong trình duyệt và quy tắc phiên bản cho từng đáp án. Khi học sinh thay đổi câu trả lời, trình duyệt tăng answer_version, ghi dữ liệu pending vào localStorage rồi gửi API. Khi mất mạng, dữ liệu pending vẫn được giữ lại; khi tải lại trang hoặc kết nối trở lại, hệ thống lấy state từ server, so sánh phiên bản và gửi lại dữ liệu chưa đồng bộ. Cách tiếp cận này hướng tới hạn chế mất bài trong các sự cố ngắn hạn, không thay thế cho các biện pháp giám sát hoặc hạ tầng mạng chuyên dụng.

1.2. Lý do chọn đề tài

Các nền tảng thi trực tuyến phổ biến thường có nhiều tính năng và phù hợp với quy mô lớn, nhưng đề tài niên luận cần một phạm vi vừa sức, có thể phân tích sâu các quy tắc nghiệp vụ. Bài toán autosave và khôi phục đáp án là một vấn đề kỹ thuật có ý nghĩa thực tế, liên quan đồng thời đến giao diện, API, giao dịch cơ sở dữ liệu, cạnh tranh request và trải nghiệm người dùng.

Đề tài phù hợp với chuyên ngành Công nghệ thông tin vì kết hợp nhiều nội dung: phát triển web phía máy chủ, thiết kế dữ liệu quan hệ, xác thực và phân quyền, bảo mật HTTP, lập trình JavaScript phía khách, kiểm thử và triển khai tài liệu kỹ thuật. Việc giới hạn hệ thống ở mô hình một giáo viên chủ hệ thống giúp tập trung vào chất lượng luồng chính thay vì mở rộng sớm sang bài toán đa tổ chức.

1.3. Mục tiêu nghiên cứu

Mục tiêu chung là xây dựng một website Toán trực tuyến có thể vận hành luồng kiểm tra từ khâu tạo đề đến công bố kết quả. Cụ thể, hệ thống cần: (1) quản lý người dùng theo hai vai trò; (2) quản lý lớp, chủ đề và ngân hàng câu hỏi; (3) hỗ trợ các loại MOT_DAP_AN, DUNG_SAI, TRA_LOI_NGAN,

TU_LUAN; (4) tạo đề nháp, giao lớp và công bố đề; (5) tạo lượt làm bài an toàn, đóng băng thứ tự câu và điểm; (6) tự động lưu, khôi phục và đồng bộ đáp án; (7) chấm tự động các câu khách quan, hỗ trợ giáo viên chấm tự luận; (8) xử lý báo cáo sự cố và bù giờ có kiểm soát.

Riêng với câu DUNG_SAI, đề tài thống nhất mô hình bốn mệnh đề a–d. Điểm của câu được xác định theo số mệnh đề chọn đúng: 1 điểm khi đúng cả bốn; 0,5 điểm khi đúng ba; 0,25 điểm khi đúng hai; 0,1 điểm khi đúng một; và 0 điểm khi không đúng mệnh đề nào. Điểm thực tế được quy đổi theo điểm tối đa của câu trong đề.

1.4. Phạm vi nghiên cứu

Về chức năng, giáo viên quản lý lớp, câu hỏi, đề thi, chấm điểm, công bố kết quả, xem thống kê và xử lý sự cố. Học sinh đăng ký, tham gia lớp bằng mã, xem đề được giao, làm bài, đánh dấu câu cần xem lại, nộp bài, xem kết quả và báo sự cố. Đề thi có vòng chờ từ NHAP đến DA_CONG_BO; chỉ đề đã công bố mới được phép bắt đầu làm. Dữ liệu seed đặt các đề ở trạng thái NHAP, do đó giáo viên cần công bố qua giao diện trước khi học sinh làm bài.

Về công nghệ, hệ thống sử dụng Node.js, Express.js, EJS, MySQL 8.0.16 trở lên, JavaScript thuần, Fetch API, localStorage, KaTeX và Chart.js. Những chức năng ngoài phạm vi gồm: nhập Word/PDF, nhập Excel, OCR, AI, camera/nhận diện khuôn mặt, chat, mobile app, WebSocket, thanh toán và admin riêng. Việc nêu rõ giới hạn giúp tránh nhầm lẫn giữa chức năng hiện có và hướng phát triển.

1.5. Đối tượng sử dụng

GIAO_VIEN là người vận hành chính: tạo lớp, quản lý học sinh trong phạm vi lớp của mình, xây dựng ngân hàng câu hỏi, tạo đề, giao đề, công bố đề, chấm bài và công bố kết quả. Trong V1, tài khoản giáo viên là tài khoản được seed hoặc tạo trong quá trình cài đặt; endpoint đăng ký công khai không cho phép tạo vai trò này.

HOC_SINH là người đăng ký công khai, tham gia lớp bằng mã hợp lệ và làm đề được phân công. Học sinh không có quyền xem đáp án đúng trong khi thi và chỉ xem kết quả khi giáo viên công bố. Hệ thống không xây dựng admin riêng bởi mô hình vận hành hiện tại chỉ có một giáo viên chủ hệ thống; thêm vai trò quản trị sẽ làm tăng phạm vi phân quyền mà không phục vụ trực tiếp mục tiêu autosave.

1.6. Phương pháp thực hiện

Đề tài bắt đầu bằng việc xác định các tác nhân, quy tắc thời gian và các rủi ro mất dữ liệu. Từ yêu cầu đó, hệ thống được thiết kế qua use case, ERD, sơ đồ kiến trúc và các luồng sequence. Giai đoạn lập trình áp dụng tách lớp Route–Controller–Service–Repository để giảm phụ thuộc giữa HTTP, nghiệp vụ và SQL. Sau cùng, các test case được lập theo từng nhóm chức năng, đặc biệt chú ý refresh, offline/online, double-click, request đến đảo thứ tự và bù giờ.

1.7. Công nghệ sử dụng

Phía giao diện sử dụng HTML5, CSS3, JavaScript thuần và EJS để render trang từ server. EJS phù hợp với hệ thống biểu mẫu quản lý vì cho phép kết hợp dữ liệu và mẫu giao diện mà không cần xây dựng SPA phức tạp. Fetch API phục vụ các API JSON của phòng thi. KaTeX render công thức Toán nhanh ở phía client; Chart.js biểu diễn phân bố điểm và tỷ lệ đúng của câu hỏi.

Phía server dùng Node.js và Express.js. Cơ sở dữ liệu MySQL 8.0.16+ được truy cập qua mysql2; mốc phiên bản này quan trọng vì CHECK constraint được MySQL thực thi từ phiên bản nêu trên. Các thư viện hỗ trợ gồm bcrypt để băm mật khẩu, express-session và express-mysql-session để quản lý phiên, csrf-csrf chống giả mạo yêu cầu, helmet thiết lập header bảo mật, express-rate-limit giới hạn tốc độ, express-validator kiểm tra dữ liệu và multer xử lý ảnh câu hỏi.

CHƯƠNG 2 — PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1. Khảo sát và phân tích yêu cầu

Yêu cầu giáo viên gồm: quản lý lớp và thành viên; quản lý chủ đề, câu hỏi và ảnh minh họa; tạo đề nháp, thêm câu, đặt điểm, giao cho lớp, công bố; theo dõi lượt làm, chấm tự luận, công bố kết quả; xem thống kê và xử lý sự cố. Giáo viên chỉ được thao tác lên tài nguyên mình sở hữu.

Yêu cầu học sinh gồm: đăng ký và đăng nhập; tham gia/rời lớp; xem đề được phân công; bắt đầu lượt làm trong thời gian hợp lệ; trả lời, đánh dấu, autosave, khôi phục; nộp bài; xem kết quả sau công bố; báo sự cố. Học sinh không thể tự bù thời gian và không được truy cập tài nguyên của người khác.

Yêu cầu phi chức năng trọng yếu là bảo mật, tính toàn vẹn, khả dụng và nhất quán thời gian. Mật khẩu không lưu dạng rõ; endpoint nhạy cảm được bảo vệ bằng session, CSRF, phân quyền và rate limit. Server đặt múi giờ +07:00/Asia/Ho_Chi_Minh làm nguồn thời gian chuẩn; đồng hồ client chỉ hiển thị theo offset thời gian máy chủ.

2.2. Mô hình Use Case

Hai actor chính là Giáo viên và Học sinh. Chuỗi nghiệp vụ cốt lõi là: giáo viên tạo lớp và câu hỏi, tạo đề NHAP, giao cho lớp, công bố thành DA_CONG_BO; học sinh bắt đầu lượt, làm bài và nộp; hệ thống chấm tự động; giáo viên chấm tự luận nếu có; giáo viên công bố kết quả; học sinh xem kết quả. Các use case hỗ trợ bao gồm autosave/khôi phục, báo sự cố, duyệt bù giờ và thống kê.

Nhóm use case giáo viên có thể ký hiệu từ UC-GV-01 đến UC-GV-14, bao phủ đăng nhập, lớp học, chủ đề, câu hỏi, đề thi, phân công, công bố, chấm, kết quả, thống kê và sự cố. Nhóm UC-HS-01 đến UC-HS-11 bao phủ đăng ký, đăng nhập, tham gia lớp, xem đề, bắt đầu, làm bài, lưu/khôi phục, nộp, xem kết quả và báo sự cố. Ma trận liên kết use case với test case được lưu trong docs/test-cases.md.

2.3. Mô tả Use Case tiêu biểu

Tạo và công bố đề: giáo viên tạo đề ở trạng thái NHAP, thêm câu hỏi, gán điểm và giao đề cho lớp. Trước khi công bố, service kiểm tra đề có câu hỏi, tổng điểm hợp lệ và điều kiện nghiệp vụ cần thiết. Công bố thành công chuyển trạng thái sang DA_CONG_BO và khóa việc thay đổi cấu trúc, tránh làm lệch dữ liệu của lượt làm đã phát sinh.

Bắt đầu và làm bài: học sinh yêu cầu bắt đầu đề đã được giao trong khung thời gian mở. Server kiểm tra thành viên lớp, trạng thái đề, số lượt cho phép và trạng thái hiện có. Trong transaction, hệ thống tạo `luot_lam_bai`, tính hạn nộp, trộn câu bằng Fisher–Yates khi cấu hình cho phép và lưu snapshot thứ tự/điểm. Vì snapshot được lưu riêng, refresh không làm thay đổi thứ tự hiển thị.

Autosave và khôi phục: mỗi thay đổi đáp án tạo phiên bản tăng dần. Client ghi `localStorage` trước khi gọi API để bảo toàn dữ liệu khi mạng lỗi. Server chỉ chấp nhận phiên bản mới hơn, có xử lý idempotent cho cùng request và từ chối request cũ bằng mã `OLD_ANSWER_VERSION`. Khi vào lại phòng thi, state server được render trước; các bản ghi local chưa đồng bộ được gửi lại. Điều này đảm bảo server vẫn là nguồn dữ liệu chính, còn `localStorage` là hàng đợi phục hồi trên thiết bị.

Chấm và công bố: khi nộp bài, câu một đáp án, đúng/sai và trả lời ngắn được chấm tự động. Câu tự luận chờ giáo viên nhập điểm và nhận xét, với kiểm tra không vượt điểm tối đa. Khi tất cả câu

tự luận đã chấm, lượt chuyên DA_CHAM. Kết quả chỉ được học sinh xem sau khi giáo viên bật công bố; đáp án chỉ được hiển thị nếu chờ cho xem đáp án được bật.

Xử lý sự cố: học sinh báo loại sự cố và mô tả; không được nhập giây bù. Giáo viên duyệt hoặc từ chối. Khi duyệt, transaction khóa bản ghi sự cố và lượt làm, cộng số giây bù một lần vào trường riêng; nếu phù hợp có thể mở lại lượt đã nộp do sự cố. Cách này chống việc duyệt lặp làm cộng thời gian nhiều lần.

2.4. Thiết kế cơ sở dữ liệu

MySQL được chọn do tính phổ biến, mô hình quan hệ rõ ràng và hỗ trợ transaction, khóa dòng, foreign key, UNIQUE, ENUM và CHECK. CSDL web_kiem_tra_toan có 15 bảng nghiệp vụ; bảng sessions do express-mysql-session tạo là bảng kỹ thuật, không thuộc ERD nghiệp vụ.

Các nhóm bảng chính gồm người dùng và lớp học (nguoi_dung, lop_hoc, thanh_vien_lop); nội dung học tập (chu_de, cau_hoi, dap_an); đề và phân công (de_thi, cau_hoi_de_thi, phan_cong_de); làm bài (luot_lam_bai, cau_hoi_luot_lam, chi_tiet_bai_lam, nhat_ky_thi); và quản lý sự cố (su_co_bai_thi). ERD chi tiết được trình bày tại diagrams/erd.md.

Khóa ngoại ghép và các quan hệ của lượt làm bảo đảm đáp án chỉ thuộc câu đã được đóng băng trong chính lượt đó. chi_tiet_bai_lam lưu câu trả lời, điểm, cờ đánh dấu và answer_version. Ràng buộc UNIQUE bảo vệ các giá trị như email hoặc mã lớp; CHECK giới hạn dữ liệu điểm, thời lượng và trạng thái hợp lệ; ENUM biểu diễn tập giá trị nghiệp vụ. Các ràng buộc database được kết hợp với validation tại service, không thay thế lẫn nhau.

2.5. Thiết kế kiến trúc hệ thống

Hệ thống sử dụng kiến trúc ba tầng: **Browser** → **Express** → **MySQL**. Browser nhận HTML do EJS render, tải JavaScript và gọi REST API cho phòng thi. Express tiếp nhận request HTTP, thực thi middleware, kiểm tra nghiệp vụ và truy cập MySQL qua connection pool. MySQL lưu dữ liệu nghiệp vụ bền vững.

Trong server, Route ánh xạ URL; Controller tiếp nhận request và chọn view/JSON; Service chứa quy tắc nghiệp vụ, phân quyền và transaction; Repository chỉ thực hiện SQL có placeholder. Middleware xử lý xác thực, CSRF, rate limit, upload, lỗi. Job tự động quét các lượt quá hạn và gọi chung logic nộp bài. Kiến trúc này giúp giảm lặp mã và giúp kiểm thử service dễ hơn.

Hệ thống có hai kênh giao tiếp. Kênh server-rendered EJS dùng cho trang đăng nhập, quản trị lớp/câu hỏi/đề, chấm và thống kê. Kênh REST JSON dùng cho phòng thi: bắt đầu lượt, lấy state, lưu đáp án, heartbeat, nộp bài. Việc tách hai kênh phù hợp với đặc tính tương tác thường xuyên của autosave mà vẫn giữ được lợi ích của EJS ở các biểu mẫu quản trị.

2.6. Thiết kế luồng xử lý chính

Đăng nhập chuẩn hóa email bằng `trim()` và `toLowerCase()`, đối chiếu `bcrypt.compare`, sau đó tái tạo session để giảm nguy cơ session fixation. Session chỉ lưu thông tin cần thiết như id, tên, email và vai trò. Middleware `requireAuth`, `requireTeacher`, `requireStudent` kiểm tra quyền trước controller.

Hạn nộp gốc của một lượt được tính theo công thức:

```
han_nop = MIN(thoi_gian_bat_dau_luot + thoi_luong_de, thoi_gian_ket_thuc_de)
```

Khi có sự cố được duyệt, hạn hiệu lực là:

```
han_nop_hieu_luc = han_nop + thoi_gian_bo_sung_giay
```

`han_nop` không thay đổi, nhờ đó có thể truy vết thời hạn gốc. Server không nhận lưu đáp án sau `han_nop_hieu_luc`. Nộp bài có ba lớp: client nộp khi đồng hồ về 0, API kiểm tra thời hạn, và job server quét lượt quá hạn mỗi khoảng 30–60 giây. Tất cả đều dùng cùng logic transaction để tránh trạng thái không nhất quán.

Heartbeat từ client chạy khoảng 30 giây để cập nhật `last_seen_at`. Nếu gián đoạn vượt ngưỡng, hệ thống có thể ghi log `MAT_KET_NOI/KHOI_PHUC` và tạo sự cố tự động khi cần. Đây là tín hiệu hỗ trợ xử lý, không phải cơ chế giám sát người học.

2.7. Thiết kế bốn loại câu hỏi

`MOT_DAP_AN` có ít nhất hai lựa chọn và chính xác một đáp án đúng. `DUNG_SAI` gồm đúng bốn mệnh đề a–d, mỗi mệnh đề có giá trị chuẩn Đúng hoặc Sai. `TRA_LOI_NGAN` không có danh sách đáp án lựa chọn mà lưu đáp án chuẩn để hệ thống tự chấm sau bước chuẩn hóa chuỗi như loại bỏ khoảng trắng dư và không phân biệt hoa/thường. `TU_LUAN` lưu nội dung trả lời nhưng không có đáp án lựa chọn; giáo viên đánh giá thủ công và nhập nhận xét.

Nội dung câu hỏi hỗ trợ biểu thức Toán qua KaTeX. Với câu có lịch sử trong đề công bố hoặc lượt làm, nội dung cũ không được sửa trực tiếp; giáo viên có thể sao chép thành câu mới. Quy tắc này bảo vệ khả năng đối chiếu bài làm với câu hỏi tại thời điểm thi.

2.8. Quy tắc thời gian và hạn nộp

Máy chủ theo múi giờ Việt Nam +07:00 là nguồn thời gian tin cậy. API state trả `serverTime`; client tính server offset và dùng offset đó cho countdown. Do đó, thay đổi đồng hồ hệ điều hành của học sinh không phải căn cứ để kéo dài thời gian làm bài. Lưu hoặc nộp bài vẫn được server kiểm tra lại độc lập, vì giao diện client không phải lớp bảo mật.

CHƯƠNG 3 — XÂY DỰNG HỆ THỐNG

3.1. Cấu trúc project và môi trường

Dự án tổ chức các thư mục `src/`, `public/`, `database/`, `docs/`, `diagrams/`, `report/` và `screenshots/`. Trong `src/` có các phần `config`, `controllers`, `middleware`, `repositories`, `routes`, `services`, `validators`, `jobs`, `utils` và `views`. `public/` chứa CSS, JavaScript phòng thi và thư viện cục bộ. `database/` chứa script reset schema, schema, seed và cập nhật liên quan.

Môi trường Node.js yêu cầu phiên bản từ 20. Các script chính là `npm run dev` để phát triển bằng nodemon, `npm start` để chạy ứng dụng, `npm test` để chạy regression và `npm run check` để quét encoding cùng render smoke-test giao diện. Biến bí mật như thông tin MySQL, `SESSION_SECRET` và SMTP nằm trong `.env`, không đưa vào báo cáo hay commit. Nếu SMTP được cấu hình, chức năng quên mật khẩu gửi mã sáu số có thời hạn; đây là tiện ích phụ trợ.

3.2. Kết nối và quản lý database

`src/config/database.js` khởi tạo connection pool mysql2, tăng khả năng phục vụ nhiều request mà không tạo kết nối mới cho từng thao tác. Múi giờ kết nối được đặt tương thích với nghiệp vụ +07:00. Các thao tác cần toàn vẹn như bắt đầu lượt, lưu đáp án, nộp bài, chấm và duyệt sự cố được đặt trong helper transaction. Nếu một bước thất bại, transaction rollback để tránh dữ liệu dở dang.

Repository dùng truy vấn có placeholder thay vì ghép chuỗi SQL từ dữ liệu người dùng. Service chịu trách nhiệm kiểm tra sở hữu, trạng thái đề, thời hạn và quy tắc nghiệp vụ trước khi gọi

repository. Sự phân chia này vừa hỗ trợ chống SQL injection vừa giúp câu lệnh SQL không bị lẫn với điều khiển HTTP.

3.3. Khởi tạo Express và middleware

app.js thiết lập EJS, static file, parser dữ liệu, method override, logging và xử lý lỗi. Session được lưu ở MySQL; cookie đặt httpOnly, sameSite: lax và chỉ đặt secure trong môi trường production HTTPS. Khi người dùng đăng nhập thành công, session được regenerate trước khi lưu thông tin người dùng.

CSRF token được đưa vào form EJS và header các request Fetch thay đổi dữ liệu. Helmet bổ sung các header bảo vệ, bao gồm CSP phù hợp với tài nguyên KaTeX và Chart.js phục vụ cục bộ. Error handler phân biệt lỗi API JSON với trang lỗi 404/500 để người dùng nhận phản hồi đúng ngữ cảnh.

3.4. Module đăng nhập và phân quyền

Đăng ký công khai chỉ tạo HOC_SINH; payload cố gắng gán GIAO_VIEN bị từ chối. Mật khẩu được kiểm tra độ dài trước khi băm bằng bcrypt. Đăng nhập xác thực email, trạng thái tài khoản và mật khẩu băm. Rate limit được áp dụng cho login và register để hạn chế thử mật khẩu hàng loạt; bộ test xác nhận ngưỡng 20 yêu cầu sai trong 15 phút trước khi trả HTTP 429.

Middleware quyền được dùng xuyên suốt: chưa đăng nhập gọi API trả 401 UNAUTHORIZED; học sinh truy cập khu vực giáo viên nhận 403 FORBIDDEN. Ngoài kiểm tra vai trò, service kiểm tra quan hệ sở hữu để một giáo viên không sửa dữ liệu của giáo viên khác.

3.5. Module lớp học, chủ đề và ngân hàng câu hỏi

Giáo viên tạo, cập nhật, lưu trữ lớp và quản lý thành viên. Mã lớp là duy nhất; học sinh dùng mã để tham gia. Việc rời lớp không xóa vật lý lịch sử và bị chặn khi học sinh đang có lượt DANG_LAM. Lớp lưu trữ không nhận thành viên mới.

Ngân hàng câu hỏi có CRUD chủ đề và bốn dạng câu hỏi đã nêu. Validation bảo đảm MOT_DAP_AN có đúng một đáp án đúng, DUNG_SAI có đủ bốn mệnh đề, trả lời ngắn có đáp án chuẩn và tự luận không bị gán đáp án lựa chọn. Ảnh câu hỏi được upload bằng Multer với giới hạn kiểu và dung lượng. KaTeX render nội dung công thức trên giao diện; câu đã có lịch sử được sao chép thay vì sửa.

3.6. Module đề thi và bắt đầu lượt làm

Đề mới được tạo ở NHAP và có thể có `tong_diem = 0` trước khi thêm câu. Giáo viên thêm/xóa/sắp xếp câu, đặt điểm và hệ thống đồng bộ tổng điểm. Khi giao đề, hệ thống tạo quan hệ `phan_cong_de` với lớp thuộc giáo viên. Chuyển sang `DA_CONG_BO` chỉ thành công nếu thỏa các quy tắc; sau đó cấu trúc đề không thể chỉnh sửa.

API bắt đầu lượt dùng transaction và khóa dữ liệu phù hợp để chống nhấp đôi hay hai request đồng thời. Sau kiểm tra quyền, thời gian, số lượt và trạng thái đề, server tạo lượt, snapshot câu hỏi/điểm trong `cau_hoi_luot_lam` và ghi nhật ký. Nếu cấu hình trộn câu, thuật toán Fisher–Yates được dùng một lần khi tạo lượt; mọi lần mở lại dùng snapshot đó.

3.7. Phòng thi, autosave và khôi phục

Phòng thi là kết hợp EJS với `exam-room.js`. API state trả câu hỏi, đáp án đã lưu, `serverTime`, hạn hiệu lực và các thông tin cần thiết; không trả cờ đáp án đúng hay đáp án ngắn chuẩn trong khi thi. Giao diện hiển thị trạng thái “Đang lưu”, “Đã lưu”, “Chờ đồng bộ” hoặc “Mất kết nối”, giúp học sinh biết dữ liệu đang ở đâu.

Mỗi thay đổi được lưu theo thứ tự: tăng `version`, ghi `localStorage` với key `math_exam_user_<userId>_attempt_<attemptId>`, đánh dấu `synced: false`, rồi gọi API. Câu một đáp án và đúng/sai gửi gần như ngay; trả lời ngắn debounce khoảng 600 ms; tự luận khoảng 1.500 ms. Khi server xác nhận, record local chuyển `synced: true`. Dữ liệu local chỉ bị xóa sau khi server xác nhận nộp bài thành công.

Server đọc `answer_version` hiện có trong transaction. Request có `version` không lớn hơn bản hiện tại bị từ chối `OLD_ANSWER_VERSION`, trừ trường hợp idempotent cùng request hợp lệ. Nhờ vậy, nếu request v2 đến trước v1, v1 không được phép ghi đè câu trả lời mới hơn. Khi refresh, client render server state trước, đọc `localStorage` và chỉ gửi lại đáp án pending; sự kiện online cũng kích hoạt đồng bộ lại.

3.8. Heartbeat, nộp bài, chấm điểm và kết quả

Heartbeat cập nhật `last_seen_at` khoảng 30 giây/lần. Nộp bài thủ công vô hiệu hóa thao tác lặp và flush các lần lưu debounce đang chờ trước khi gửi submit. Với hết giờ, client chủ động nộp; nếu client không còn hoạt động, `auto-submit-job.js` quét server để nộp các lượt quá hạn. Service

nộp bài khóa lượt làm, chỉ cho phép chuyển từ DANG_LAM, chấm câu khách quan, cập nhật trạng thái DA_NOP hoặc TU_DONG_NOP và ghi nhật ký.

Chấm một đáp án, đúng/sai và trả lời ngắn được thực hiện tự động. Với tự luận, giáo viên nhập điểm không vượt mức điểm câu và nhận xét. Tổng điểm được tính lại từ chi tiết thay vì cộng dồn nhằm tránh tăng điểm sau các lần chấm lại. Nếu đề không có tự luận, lượt có thể đạt DA_CHAM ngay sau nộp; nếu có, trạng thái chỉ hoàn tất khi chấm hết. Kết quả không thể xem trước thời điểm công bố; đáp án chỉ hiện khi cờ cho_xem_dap_an cho phép.

3.9. Module xử lý sự cố và thống kê

Học sinh có thể báo mất mạng hoặc sự cố liên quan trong lúc làm bài. Giáo viên xem danh sách, duyệt hoặc từ chối. Duyệt bù giờ thực hiện trong transaction, khóa sự cố và lượt, chỉ cộng thời_gian_bo_sung_giay một lần. Trường hợp bài tự nộp do sự cố có thể được mở lại theo quy tắc nghiệp vụ và được ghi log MO_LAI_SAU_SU_CO.

Thống kê cung cấp số lượt đã làm/chưa làm, điểm trung bình/cao/thấp, phân bố điểm và tỷ lệ đúng theo câu. Chart.js được dùng để trực quan hóa các số liệu này. Các biểu đồ có vai trò hỗ trợ giáo viên quan sát kết quả lớp, không được xem là công cụ phân tích học tập chuyên sâu.

3.10. Bảo mật

Các biện pháp chính gồm bcrypt cho mật khẩu, session MySQL, regenerate session khi đăng nhập, CSRF, Helmet, rate limit, validation dữ liệu, SQL placeholder và kiểm tra quyền sở hữu. Upload được giới hạn để giảm rủi ro từ tệp không phù hợp. API phòng thi ẩn đáp án đúng trong lúc thi. Những biện pháp này giảm các rủi ro phổ biến, nhưng khi triển khai thực tế vẫn cần HTTPS, quản lý secrets, sao lưu, giám sát máy chủ và quy trình cập nhật thư viện.

CHƯƠNG 4 — KIỂM THỬ VÀ ĐÁNH GIÁ

4.1. Môi trường kiểm thử

Kiểm thử thực hiện trên môi trường local tại `http://localhost:3000`, Node.js LTS (dự án yêu cầu Node.js từ 20) và MySQL 8.0.16 trở lên. Dữ liệu seed gồm giáo viên `teacher@example.com`, học sinh `studenta@example.com`, `studentb@example.com`; cả ba dùng mật khẩu demo 123456. Dữ liệu

còn có lớp TOAN10A1, bốn câu hỏi thuộc bốn loại, đề 15 phút tổng 5,00 điểm và phân công demo. Tài khoản demo chỉ phục vụ cài đặt/kiểm thử, không phù hợp dùng ở môi trường thật.

4.2. Bộ test case

Tài liệu docs/test-cases.md định nghĩa **99 test case thủ công**. Các nhóm gồm tài khoản (TC-ACC), lớp học (TC-CLS), câu hỏi/chủ đề (TC-QST), đề thi (TC-EXM), làm bài (TC-TAK), chấm điểm/kết quả (TC-GRD), sự cố (TC-INC) và hệ thống (TC-SYS). Mỗi test case quy định điều kiện trước, dữ liệu đầu vào, bước thao tác và kết quả mong đợi để có thể lặp lại.

Bên cạnh kiểm thử thủ công, lệnh `npm test` chạy bộ regression tự động với kết quả **41/41**. Regression tập trung bảo vệ các quy tắc đã hiện thực để thay đổi mã nguồn sau này không làm hỏng các hành vi cốt lõi (gồm CSRF multipart, cleanup file upload, guard xóa câu/đề). Lệnh `npm run check` bổ sung quét encoding và render smoke-test các giao diện chính.

4.3. Kết quả kiểm thử

Các nhóm test bao phủ luồng chính từ đăng ký học sinh, đăng nhập, quản lý lớp/câu hỏi/đề đến bắt đầu làm bài, autosave, nộp, chấm, công bố và xử lý sự cố. Với dữ liệu seed, đề cần được giáo viên công bố từ NHAP sang DA_CONG_BO trước khi chạy nhóm test làm bài; đây là điều kiện nghiệp vụ, không phải lỗi seed.

Kết quả regression tự động xác nhận các kiểm tra đã chạy thành công tại thời điểm lập báo cáo. Danh sách 99 trường hợp thủ công là tài liệu kiểm thử chi tiết; khi nghiệm thu thực tế, cột “Kết quả thực tế” và “Đạt/Không đạt” cần được ghi nhận theo từng vòng chạy. Ảnh minh chứng giao diện được tham chiếu tại thư mục screenshots/ (18 ảnh, từ 01-trang-chu.png đến 18-hs-phong-thi.png).

4.4. Kiểm thử các tình huống đặc biệt

Mất mạng và khôi phục: khi đặt trình duyệt Offline, câu trả lời mới được lưu pending ở localStorage. Khi Online lại, event đồng bộ gửi các bản ghi chưa xác nhận. Trường hợp refresh sau khi đã lưu hoặc đang có pending, state server và local được so sánh; thứ tự câu không thay đổi do snapshot đã được tạo lúc bắt đầu lượt.

Cạnh tranh request: test TC-TAK-10 và TC-TAK-22 kiểm tra version cũ và hai request đến đảo thứ tự. Nếu DB đã có version 3 mà client gửi version 1, server trả OLD_ANSWER_VERSION và không ghi

đề. Nếu v2 đến trước v1, đáp án B của v2 vẫn được giữ; client không được tùy tiện tăng version để gửi lại payload cũ.

Nộp hai lần và bắt đầu hai lần: submit lặp bị chặn do trạng thái lượt không còn DANG_LAM; start dùng transaction/ khóa phù hợp để một request tạo lượt, request còn lại nhận trạng thái đúng thay vì tạo bản ghi trùng. Test cũng kiểm tra trường hợp nhập tự luận rồi nộp ngay, trong đó client flush save đang chờ trước submit.

Hết giờ và bù giờ: client tự nộp khi countdown về 0, API chặn lưu sau hạn, job server làm tăng dự phòng khi client đóng. Khi giáo viên duyệt sự cố, thời hạn hiệu lực tăng đúng số giây duyệt; duyệt lần hai bị từ chối để không cộng trùng. Phân quyền sai vai trò hoặc sai quyền sở hữu đều phải trả lỗi và không thay đổi dữ liệu.

4.5. Kịch bản demo

Kịch bản demo 10–15 phút được lưu tại docs/demo-script.md. Phần mở đầu đăng nhập bằng giáo viên, giới thiệu lớp, ngân hàng bốn loại câu và đề nháp. Giáo viên công bố đề và học sinh đăng nhập bằng tài khoản demo để tham gia hoặc xem lớp, bắt đầu đề. Trong phòng thi, người trình bày trả lời một vài câu, quan sát trạng thái autosave, refresh trang để chứng minh khôi phục và có thể mô phỏng offline/online. Cuối cùng, học sinh nộp bài, giáo viên chấm tự luận nếu có, công bố kết quả và xem thống kê.

Các ảnh cần chèn khi xuất PDF gồm: 01-trang-chu.png, các ảnh giao diện đăng nhập/quản lý của giáo viên, giao diện phòng thi, autosave/khôi phục, kết quả và 12-hs-de-thi.png. Ảnh nên có chú thích “Hình x.y” và nêu rõ chức năng quan sát được, thay vì chỉ mô tả giao diện.

4.6. Đánh giá hệ thống

Ưu điểm quan trọng nhất của JinMath là autosave đa tầng, có answer_version để hạn chế ghi đè do request cũ và localStorage để hỗ trợ gián đoạn tạm thời. Hệ thống hỗ trợ đủ bốn loại câu hỏi trong phạm vi đề tài, biểu diễn công thức bằng KaTeX, chấm tự động câu khách quan, có quy trình chấm tự luận và xử lý sự cố rõ ràng. Kiến trúc phân tầng, transaction ở các luồng nhạy cảm và tài liệu test/API/ERD giúp dự án dễ kiểm tra và bảo trì hơn.

Hạn chế của phiên bản hiện tại là chưa có ứng dụng mobile; giao diện responsive mới ở mức cơ bản; không có thông báo realtime qua WebSocket; chấm tự luận hoàn toàn thủ công và không tích hợp AI. Hệ thống cũng chưa nhằm phục vụ nhiều trường hoặc nhiều giáo viên độc lập như một

nền tảng SaaS. Các giới hạn này phù hợp với phạm vi đã khóa, cần được hiểu là hướng mở rộng chứ không phải tính năng đã có.

So với mục tiêu ban đầu, dự án đáp ứng chuỗi giáo viên tạo/giao/công bố đề, học sinh làm bài với autosave–khôi phục, nộp/chấm/công bố kết quả, và xử lý sự cố bù giờ. Kết quả 41/41 regression cùng 99 test case thủ công được chuẩn bị cho thấy yêu cầu được kiểm thử có cấu trúc. Độ tin cậy khi triển khai thực tế còn phụ thuộc vào cấu hình server, HTTPS, sao lưu cơ sở dữ liệu và chất lượng đường truyền.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Đề tài đã xây dựng JinMath, website kiểm tra Toán trực tuyến cho mô hình một giáo viên và các lớp học của mình. Hệ thống triển khai quản lý lớp, ngân hàng câu hỏi, đề thi, phòng thi, chấm điểm, công bố kết quả, thống kê và xử lý sự cố. Đóng góp kỹ thuật nổi bật là cơ chế autosave kết hợp server, localStorage và answer_version, cùng với cơ chế khôi phục khi refresh hoặc mất mạng tạm thời. Thời hạn theo server, snapshot thứ tự câu, transaction và quy tắc phân quyền hỗ trợ giữ tính nhất quán của lượt làm.

Sản phẩm có giá trị như một mô hình thực hành về xây dựng ứng dụng web có trạng thái phức tạp và yêu cầu bảo toàn dữ liệu. Dù không thay thế cho một hệ thống thi quy mô lớn, JinMath thể hiện cách kết hợp các kỹ thuật web hiện đại để giải quyết bài toán mất bài trong bối cảnh kiểm tra trực tuyến.

Trong tương lai có thể phát triển ứng dụng di động bằng React Native hoặc Flutter, tăng chất lượng responsive, bổ sung thông báo realtime bằng WebSocket, nhập câu hỏi từ Excel, mở rộng cho các môn học khác và nghiên cứu AI hỗ trợ chấm tự luận. Các hướng này là đề xuất sau niên luận, không thuộc chức năng của phiên bản hiện tại. Nếu mở rộng quy mô, cần bổ sung đa giáo viên/đa tổ chức, phân quyền quản trị, quan sát hệ thống, sao lưu và chiến lược triển khai bảo mật.

TÀI LIỆU THAM KHẢO

1. Node.js Foundation, “Node.js Documentation”, <https://nodejs.org/docs/latest/api/> (truy cập tháng 08/2026).
2. Express.js, “Express – Node.js web application framework”, <https://expressjs.com/> (truy cập tháng 08/2026).
3. Oracle, “MySQL 8.0 Reference Manual”, <https://dev.mysql.com/doc/refman/8.0/en/> (truy cập tháng 08/2026).

4. Mozilla Developer Network, “Window: localStorage property”, <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage> (truy cập tháng 08/2026).
5. KaTeX Contributors, “KaTeX Documentation”, <https://katex.org/docs/> (truy cập tháng 08/2026).
6. Chart.js Contributors, “Chart.js Documentation”, <https://www.chartjs.org/docs/latest/> (truy cập tháng 08/2026).
7. OWASP Foundation, “Authentication Cheat Sheet” và “Cross Site Request Forgery Prevention Cheat Sheet”, <https://cheatsheetseries.owasp.org/> (truy cập tháng 08/2026).
8. Bộ Giáo dục và Đào tạo, các văn bản và tài liệu hướng dẫn về ứng dụng công nghệ thông tin, chuyển đổi số trong giáo dục và tổ chức kiểm tra, đánh giá.
9. Tài liệu nội bộ dự án JinMath: README.md, docs/scope.md, docs/architecture.md, docs/api.md, docs/service-rules.md, docs/test-cases.md, diagrams/erd.md.

PHỤ LỤC

Phụ lục A — Hướng dẫn cài đặt và chạy hệ thống

Xem README.md tại thư mục gốc. Tài liệu hướng dẫn cài Node.js, MySQL, tạo .env, chạy lần lượt các script database và khởi động bằng npm run dev. README cũng liệt kê các tài khoản demo, yêu cầu MySQL 8.0.16+ và lưu ý không đưa thông tin bí mật vào mã nguồn.

Phụ lục B — Tài liệu API

Xem docs/api.md. Tài liệu mô tả các endpoint REST dùng cho phòng thi, gồm bắt đầu lượt, lấy state, lưu đáp án, heartbeat, nộp bài và các mã lỗi nghiệp vụ như OLD_ANSWER_VERSION, DEADLINE_PASSED hoặc ALREADY_SUBMITTED.

Phụ lục C — ERD và kiến trúc

Xem diagrams/erd.md để tham khảo 15 bảng nghiệp vụ, khóa chính/khóa ngoại và quan hệ dữ liệu. Xem docs/architecture.md để tham khảo sơ đồ ba tầng, luồng đăng nhập, tạo đề, bắt đầu lượt, autosave, khôi phục, nộp bài, sự cố và chấm điểm.

Phụ lục D — Ảnh giao diện

Ảnh minh họa được đặt tại `screenshots/`, sử dụng các tên từ `01-trang-chu.png` đến `18-hs-phong-thi.png`. Khi dàn trang PDF, chèn ảnh vào đúng phần mô tả chức năng và ghi chú nguồn là “Ảnh chụp từ hệ thống JinMath”.

Phụ lục E — Bộ kiểm thử

Xem `docs/test-cases.md` với 99 test case thủ công và ma trận phủ use case. Kết quả regression tự động được chạy bằng `npm test`, ghi nhận 41/41 tại thời điểm hoàn thiện báo cáo.